

Lab 7.1. Stages of the activity lifecycle

1. Welcome
2. App overview
3. Explore the lifecycle methods and add basic logging
4. Explore lifecycle use cases
5. Explore configuration changes
6. Summary
7. Learn more

[1. Welcome](#)

Introduction

In this codelab, you learn more about a fundamental part of Android: the *activity*. The *activity lifecycle* is the set of states an activity can be in during its lifetime. The lifecycle extends from when the activity is initially created to when it is destroyed and the system reclaims that activity's resources. As a user navigates between activities in your app (and into and out of your app), those activities each transition between different states in the activity lifecycle.

As an Android developer, you need to understand the activity lifecycle. If your activities do not correctly respond to lifecycle state changes, your app could generate strange bugs, confusing behavior for your users, or use too many Android system resources. Understanding the Android lifecycle, and responding correctly to lifecycle state changes, is critical to being a good Android citizen.

What you should already know

- What an activity is, and how to create one in your app.
- What the activity's `onCreate()` method does, and the kind of operations that are performed in that method.

What you'll learn

- How to print logging information to the Logcat.
- The basics of the `Activity` lifecycle, and the callbacks that are invoked when the activity moves between states.
- How to override lifecycle callback methods to perform operations at different times in the activity lifecycle.

What you'll do

- Modify a starter app called DessertClicker to add logging information that's displayed in the Logcat.
- Override lifecycle callback methods and log changes to the activity state.
- Run the app and note the logging information that appears as the activity is started, stopped, and resumed.
- Implement the `onSaveInstanceState()` method to retain app data that may be lost if the device configuration changes. Add code to restore that data when the app starts again.

2. App overview

In this codelab, you work with a starter app called DessertClicker. In this app, each time the user taps a dessert on the screen, the app "purchases" the dessert for the user. The app updates values in the layout for the number of desserts that were purchased, and for the total amount the user spent.

11:16



Dessert Clicker



Today's Special



Desserts Sold

16

\$135

This app contains several bugs related to the Android lifecycle: For example, in certain circumstances, the app resets the dessert values to 0. Understanding the Android lifecycle will help you understand why these problems happen, and how to fix them.

Get the starter app

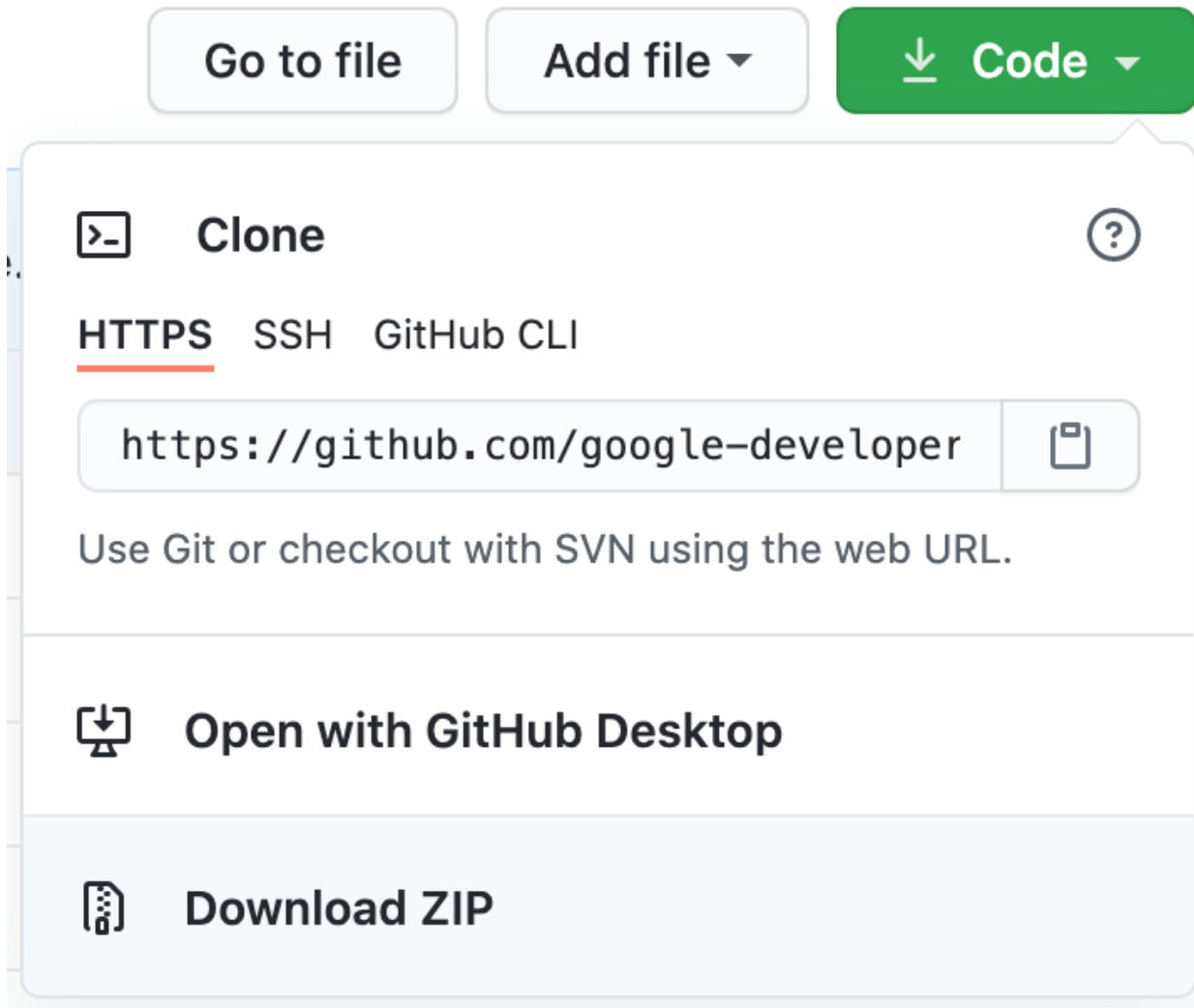
Download the [DessertClicker starter code](#) and open it in Android Studio.

If you use the starter code from GitHub, note that the folder name is `android-basics-kotlin-dessert-clicker-app-starter`. Select this folder when you open the project in Android Studio.

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

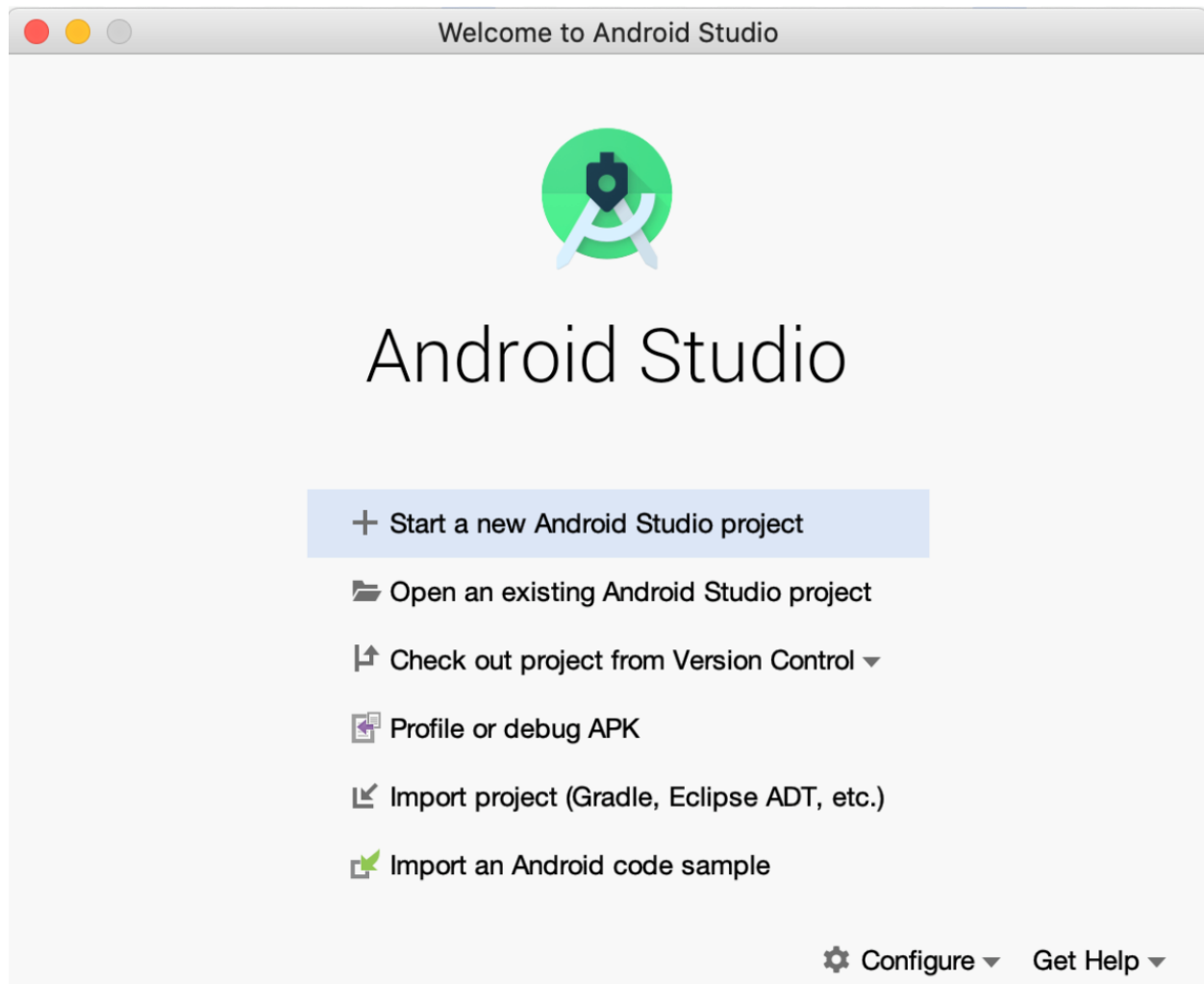
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



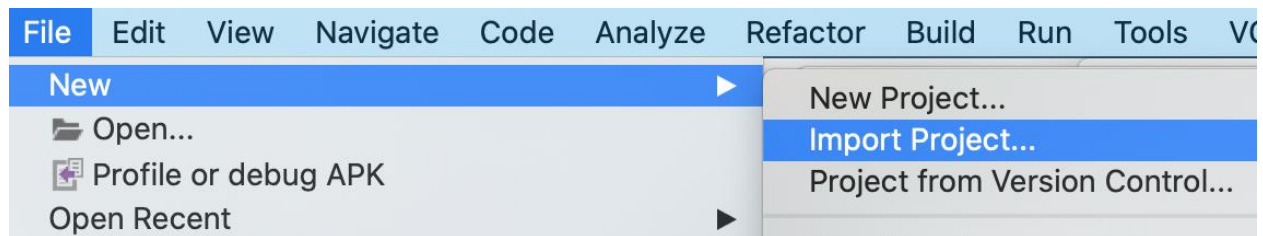
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

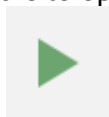
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.

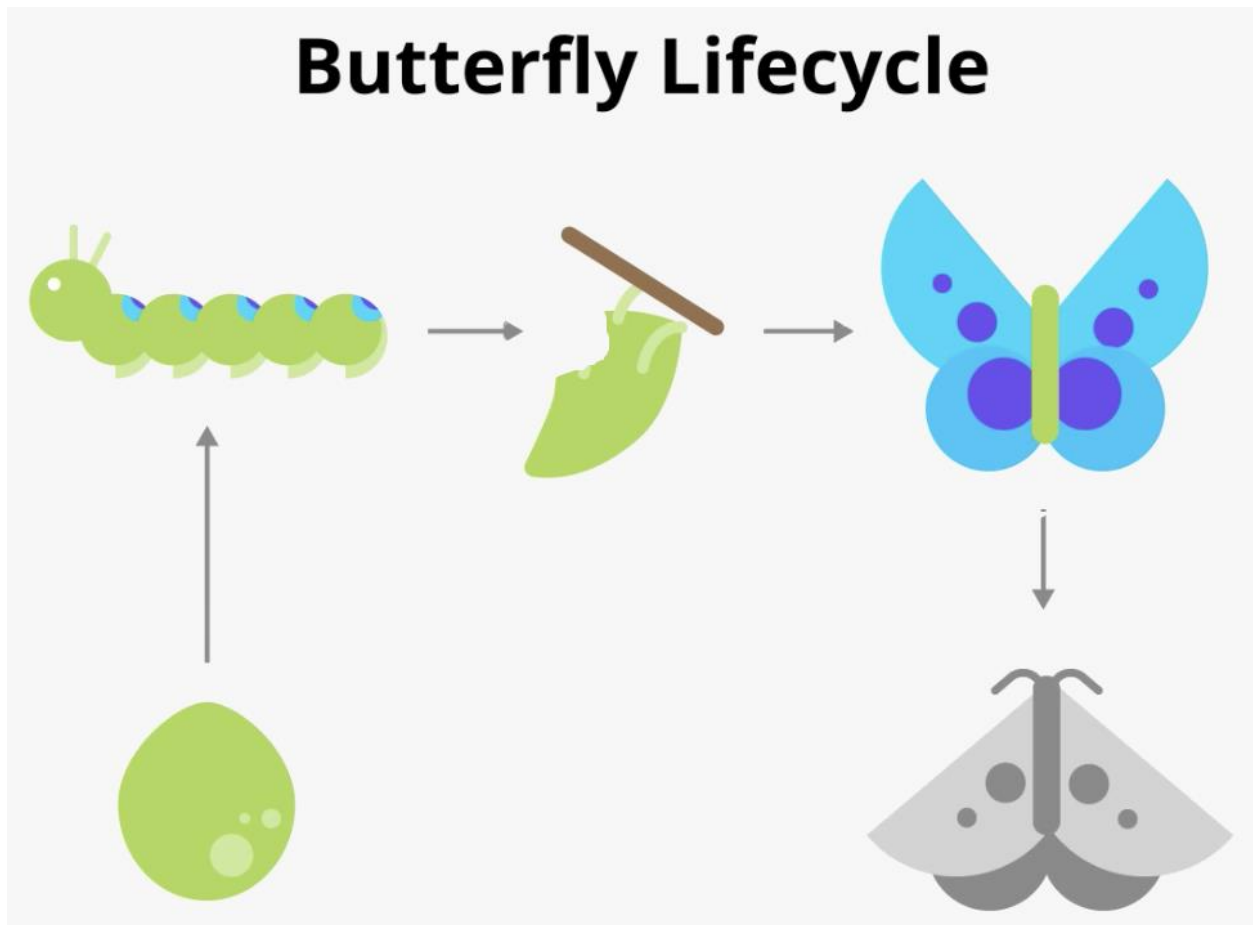


6. Click the **Run** button to build and run the app. Make sure it builds as expected.

7. Browse the project files in the **Project** tool window to see how the app is set-up.

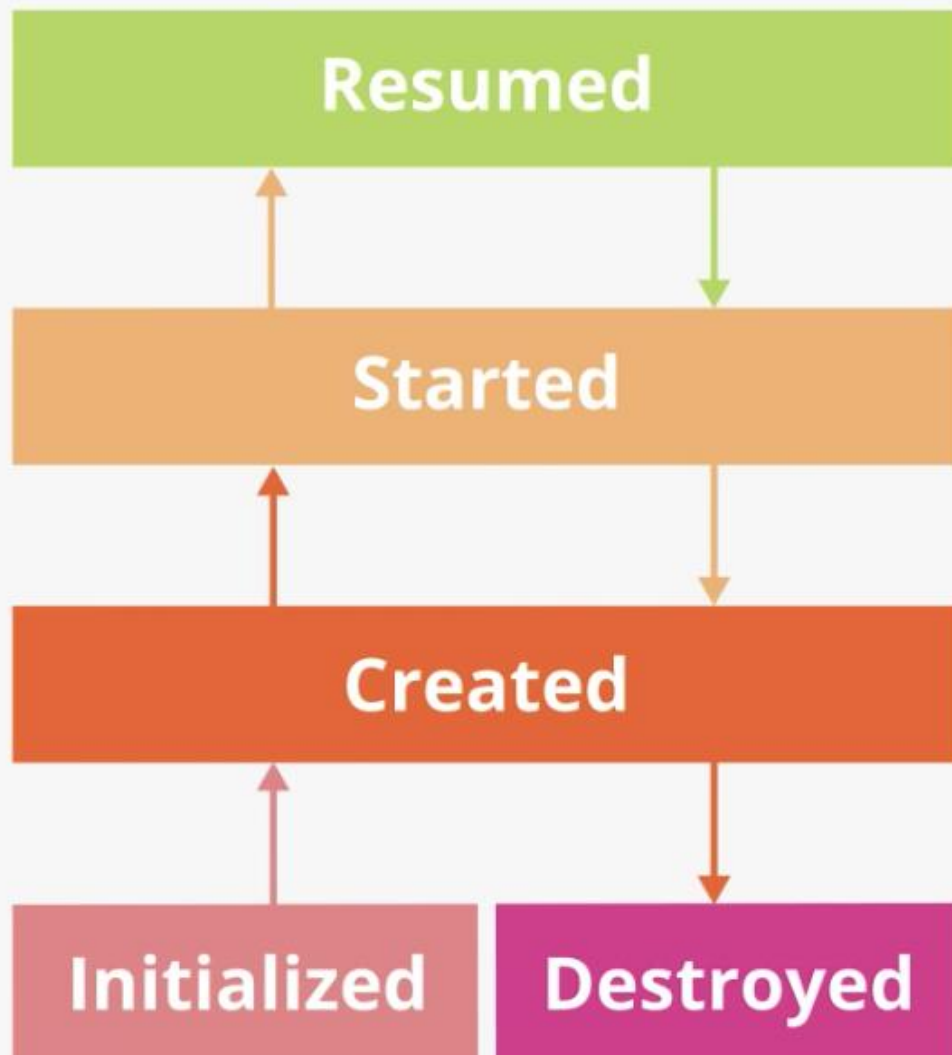
3. Explore the lifecycle methods and add basic logging

Every activity has what is known as a *lifecycle*. This is an allusion to plant and animal lifecycles, like the lifecycle of this butterfly—the different states of the butterfly show its growth from birth to fully formed adulthood to death.



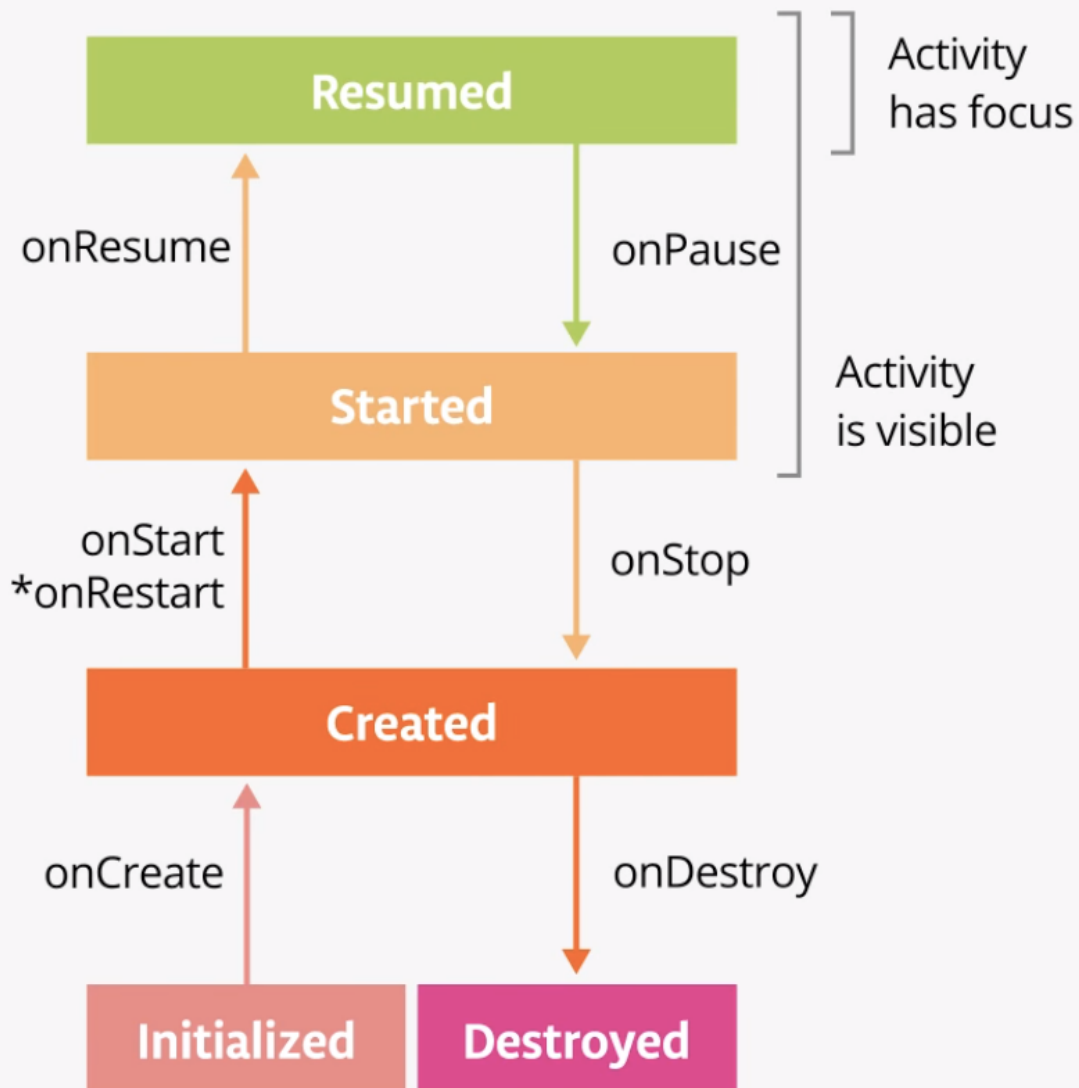
Similarly, the activity lifecycle is made up of the different states that an activity can go through, from when the activity is first initialized to when it is finally destroyed and its memory reclaimed by the system. As the user starts your app, navigates between activities, navigates inside and outside of your app, the activity changes state. The diagram below shows all the activity lifecycle states. As their names indicate, these states represent the status of the activity.

The Activity Lifecycle



Often, you want to change some behavior, or run some code when the activity lifecycle state changes. Therefore the `Activity` class itself, and any subclasses of `Activity` such as `AppCompatActivity`, implement a set of lifecycle callback methods. Android invokes these callbacks when the activity moves from one state to another, and you can override those methods in your own activities to perform tasks in response to those lifecycle state changes. The following diagram shows the lifecycle states along with the available overridable callbacks.

The Activity Lifecycle



It's important to know when these callbacks are invoked and what to do in each callback method. But both of these diagrams are complex and can be confusing. In this codelab, instead of just reading what each state and callback means, you're going to do some detective work and build your understanding of what's going on.

Step 1: Examine the `onCreate()` method and add logging

To figure out what's going on with the Android lifecycle, it's helpful to know when the various lifecycle methods are called. This will help you hunt down where things are going wrong in `DessertClicker`.

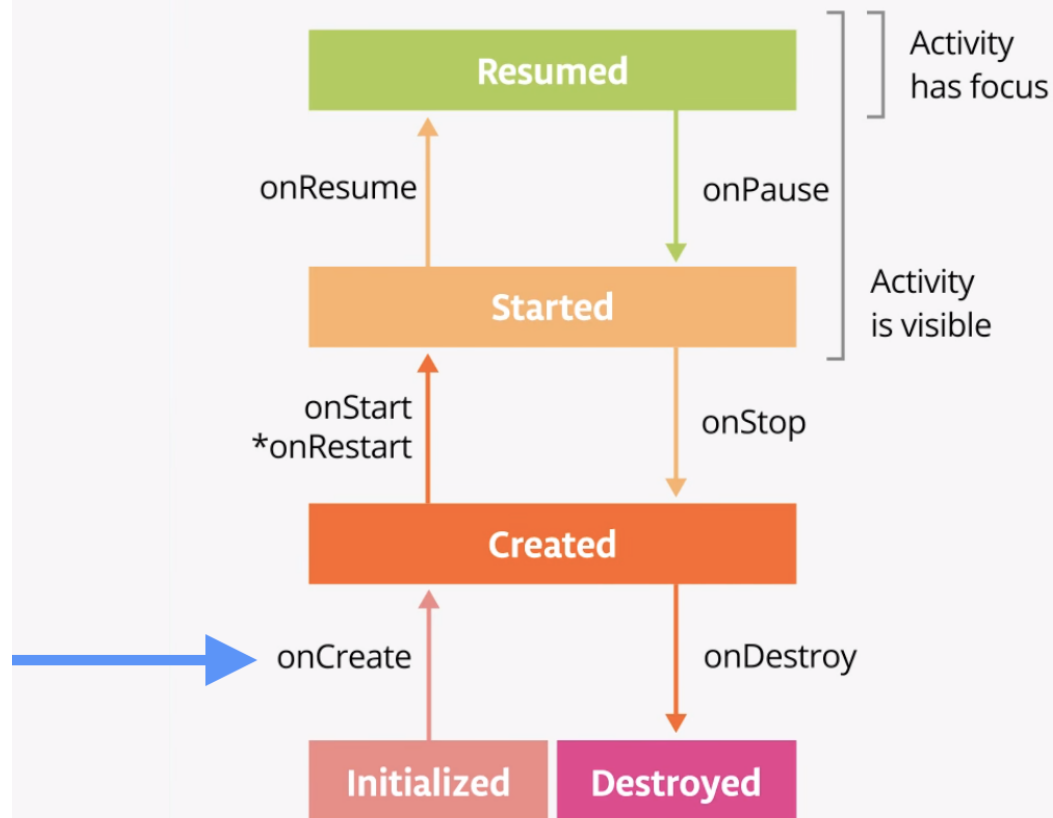
A simple way to do that is to use the Android logging functionality. Logging enables you to write short messages to a console while the app runs, and you can use it to show you when different callbacks are triggered.

1. Run the Dessert Clicker app, and tap several times on the picture of the dessert. Note how the value for **Desserts Sold** and the total dollar amount changes.
2. Open `MainActivity.kt` and examine the `onCreate()` method for this activity:

```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
}
```

In the activity lifecycle diagram, you may have recognized the `onCreate()` method, because you've used this callback before. It's the one method every activity must implement. The `onCreate()` method is where you should do any one-time initializations for your activity. For example, in `onCreate()` you inflate the layout, define click listeners, or set up view binding.

The Activity Lifecycle



The `onCreate()` lifecycle method is called once, just after the activity is initialized (when the new `Activity` object is created in memory). After `onCreate()` executes, the activity is considered *created*.

Note: When you override the `onCreate()` method, you must call the superclass implementation to complete the creation of the `Activity`, so within it, you must immediately call `super.onCreate()`. The same is true for other lifecycle callback methods.

3. In the `onCreate()` method, just after the call to `super.onCreate()`, add the following line:

```
Log.d("MainActivity", "onCreate Called")
```

4. Import the `Log` class if necessary (press `Alt+Enter`, or `Option+Enter` on a Mac, and select **Import**.) If you enabled auto imports, this should happen automatically.

```
import android.util.Log
```

The [Log](#) class writes messages to the **Logcat**. The **Logcat** is the console for logging messages. Messages from Android about your app appear here, including the messages you explicitly send to the log with the `Log.d()` method or other `Log` class methods.

There are three parts to this command:

- The *priority* of the log message, that is, how important the message is. In this case, the [Log.d\(\)](#) method writes a debug message. Other methods in the `Log` class include [Log.i\(\)](#) for informational messages, [Log.e\(\)](#) for errors, [Log.w\(\)](#) for warnings, or [Log.v\(\)](#) for verbose messages.
- The log *tag* (the first parameter), in this case `"MainActivity"`. The tag is a string that lets you more easily find your log messages in the Logcat. The tag is typically the name of the class.
- The actual log *message* (the second parameter), is a short string, which in this case is `"onCreate called"`.

Note: A good convention is to declare a TAG constant in your class:

```
const val TAG = "MainActivity"
```

and use that in subsequent calls to the log methods, like below:

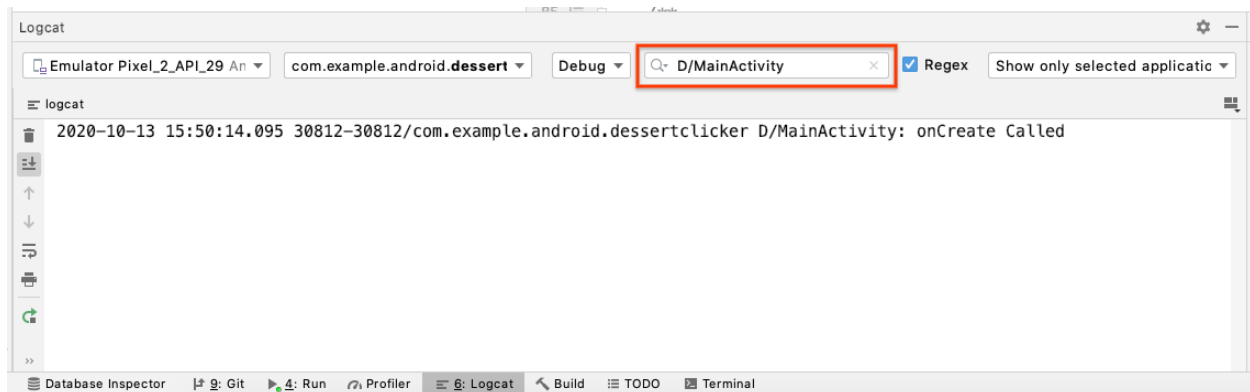
```
Log.d(TAG, "onCreate Called")
```

A [compile-time constant](#) is a value that won't change. Use `const` before a variable declaration to mark it as a compile-time constant.

5. Compile and run the DessertClicker app. You don't see any behavior differences in the app when you tap the dessert. In Android Studio, at the bottom of the screen, click the **Logcat** tab.



6. In the **Logcat** window, type `D/MainActivity` into the search field.



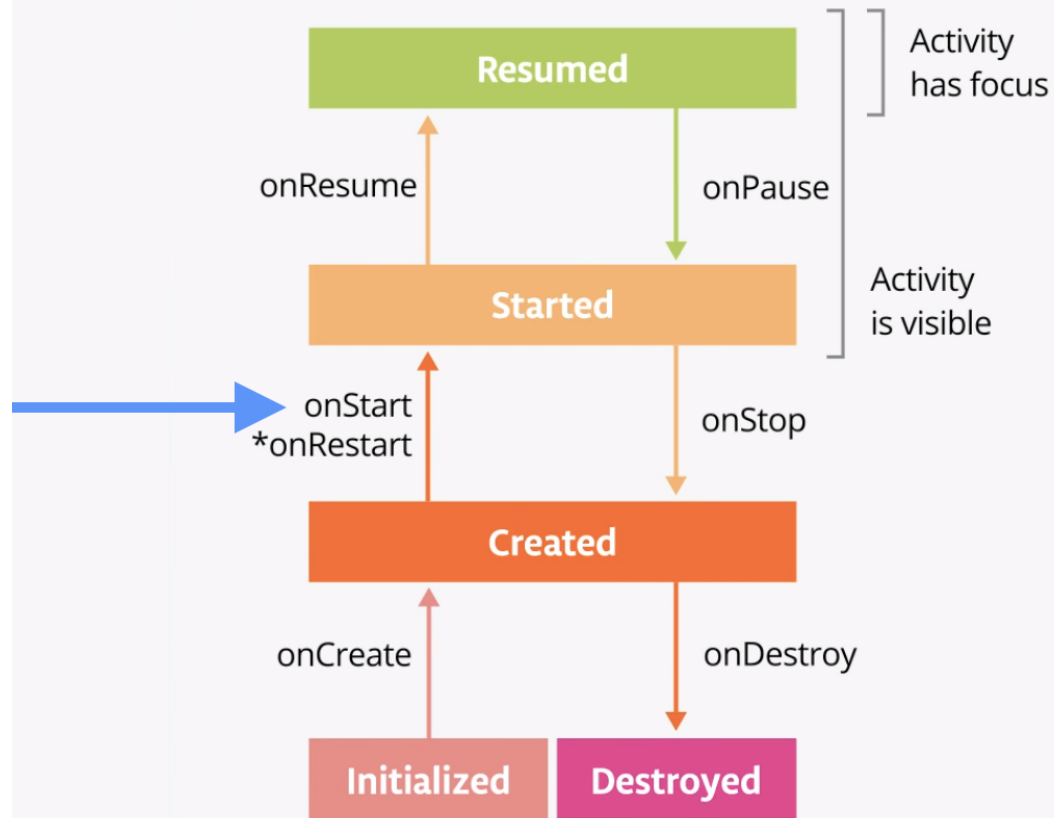
The Logcat can contain many messages, most of which aren't useful to you. You can filter the Logcat entries in many ways, but searching is the easiest. Because you used `MainActivity` as the log tag in your code, you can use that tag to filter the log. Adding `D/` at the start means that this is a debug message, created by `Log.d()`.

Your log message includes the date and time, the name of the package (`com.example.android.dessertclicker`), your log tag (with `D/` at the start), and the actual message. Because this message appears in the log, you know that `onCreate()` has been executed.

Step 2: Implement the `onStart()` method

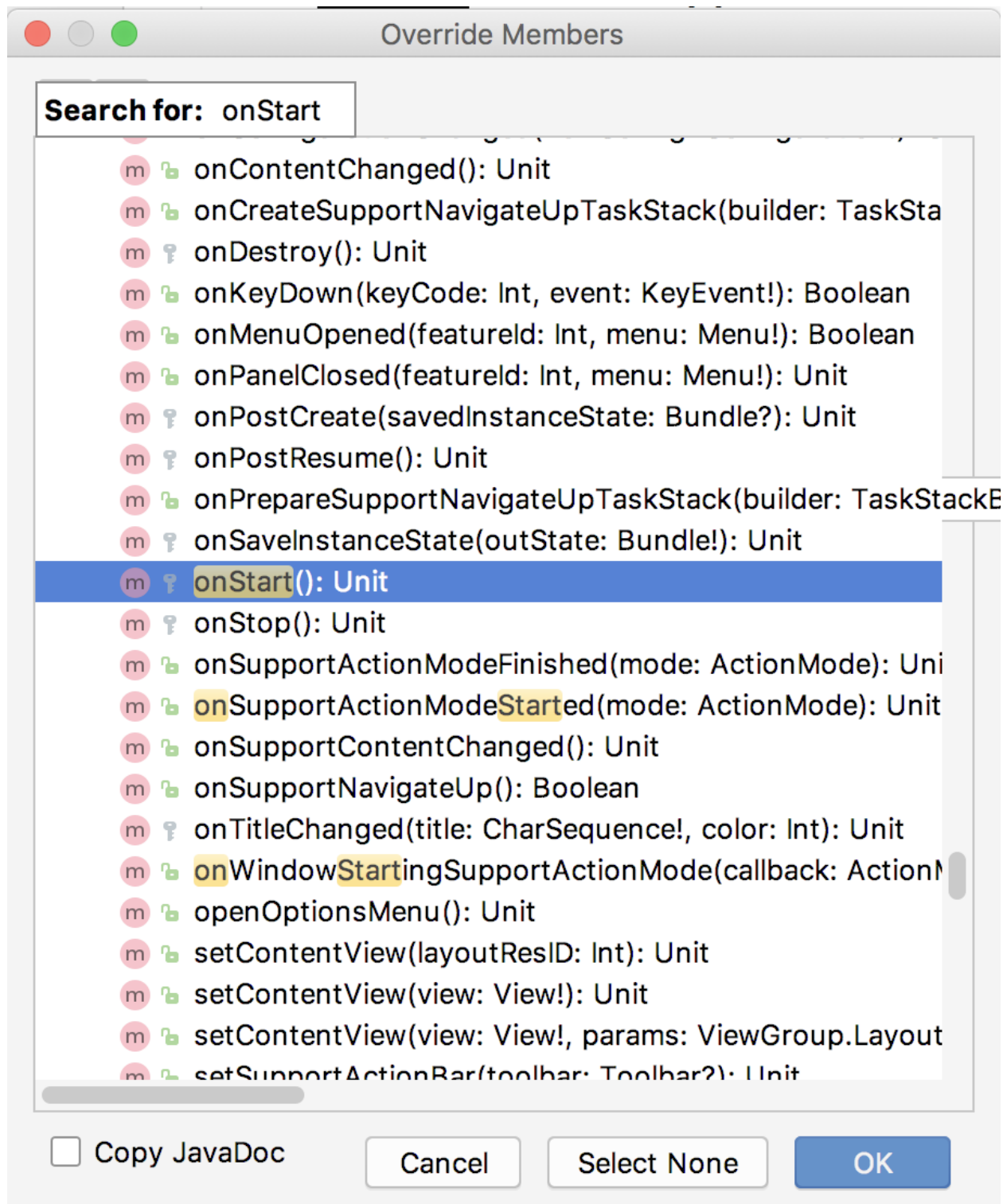
The `onStart()` lifecycle method is called just after `onCreate()`. After `onStart()` runs, your activity is visible on the screen. Unlike `onCreate()`, which is called only once to initialize your activity, `onStart()` can be called many times in the lifecycle of your activity.

The Activity Lifecycle



Note that `onStart()` is paired with a corresponding `onStop()` lifecycle method. If the user starts your app and then returns to the device's home screen, the activity is stopped and is no longer visible on screen.

1. In Android Studio, with `MainActivity.kt` open and the cursor within the `MainActivity` class, select **Code > Override Methods** or press `Control+o` (`Command+o` on Mac). A dialog appears with a huge list of all the methods you can override in this class.



2. Start entering `onStart` to search for the right method. To scroll to the next matching item, use the down arrow. Choose `onStart()` from the list, and click **OK** to insert the boilerplate override code. The code looks like this:

```
override fun onStart() {  
    super.onStart()  
}
```

3. Add the following constant at the top level of the `MainActivity.kt`, that is above the class declaration, `class MainActivity`.

```
const val TAG = "MainActivity"
```

4. Inside the `onStart()` method, add a log message:

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, "onStart Called")  
}
```

5. Compile and run the DessertClicker app, and open the **Logcat** pane. Type `D/MainActivity` into the search field to filter the log. Notice that both the `onCreate()` and `onStart()` methods were called one after the other, and that your activity is visible on screen.
6. Press the Home button on the device, and then use the recents screen to return to the activity. Notice that the activity resumes where it left off, with all the same values, and that `onStart()` is logged a second time to Logcat. Notice also that the `onCreate()` method is usually not called again.

```
16:19:59.125 31107-31107/com.example.android.dessertclicker D/MainActivity:  
onCreate Called  
16:19:59.372 31107-31107/com.example.android.dessertclicker D/MainActivity:  
onStart Called  
16:20:11.319 31107-31107/com.example.android.dessertclicker D/MainActivity:  
onStart Called
```

Note: As you experiment with your device and observe the lifecycle callbacks, you might notice unusual behavior when you rotate your device. You'll learn about that behavior later in this codelab.

Step 3: Add more log statements

In this step, you implement logging for all the other lifecycle methods.

1. Override the remainder of the lifecycle methods in your `MainActivity`, and add log statements for each one. Here's the code:

```
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, "onResume Called")  
}  
  
override fun onPause() {
```

```

        super.onPause()
        Log.d(TAG, "onPause Called")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop Called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy Called")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, "onRestart Called")
    }
}

```

2. Compile and run DessertClicker again and examine Logcat. This time notice that in addition to `onCreate()` and `onStart()`, there's a log message for the `onResume()` lifecycle callback.

```

2020-10-16 10:27:33.244 22064-22064/com.example.android.dessertclicker
D/MainActivity: onCreate Called
2020-10-16 10:27:33.453 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStart Called
2020-10-16 10:27:33.454 22064-22064/com.example.android.dessertclicker
D/MainActivity: onResume Called

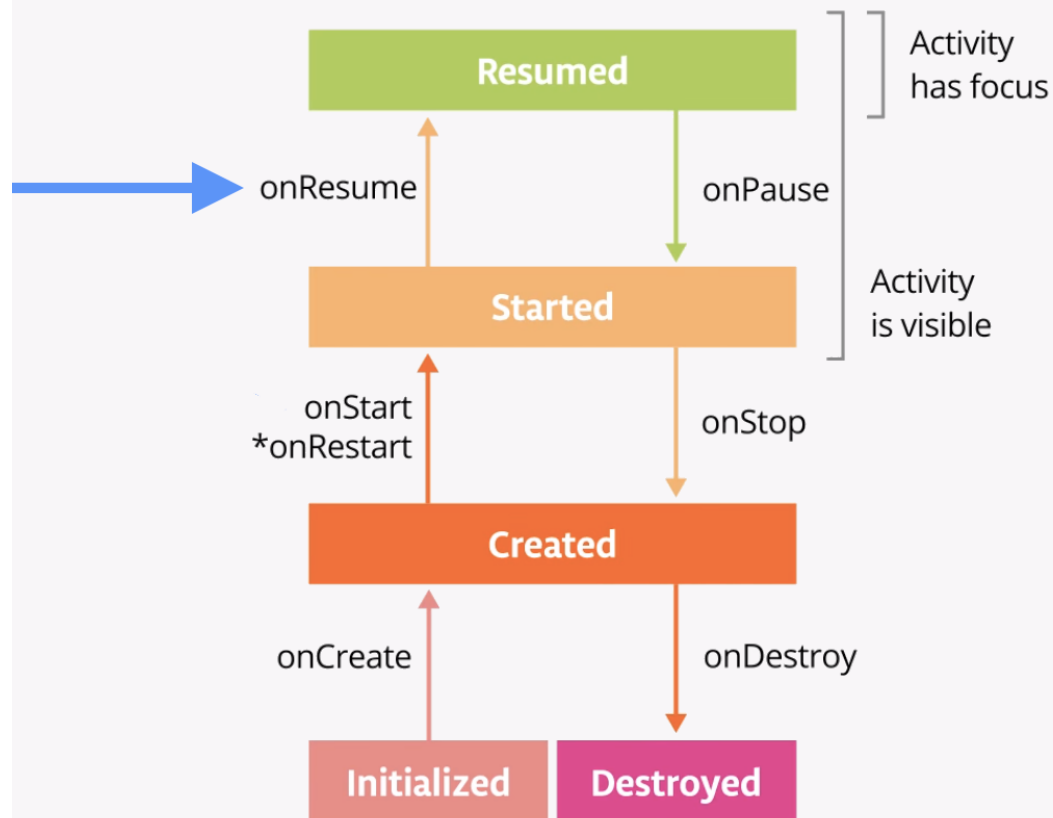
```

When an activity starts from scratch, you see all three of these lifecycle callbacks called in order:

- `onCreate()` to create the app.
- `onStart()` to start it and make it visible on the screen.
- `onResume()` to give the activity focus and make it ready for the user to interact with it.

Despite the name, the `onResume()` method is called at startup, even if there is nothing to resume.

The Activity Lifecycle



4. Explore lifecycle use cases

Now that the DessertClicker app is set up for logging, you're ready to start using the app in various ways, and ready to explore how the lifecycle callbacks are triggered in response to those uses.

Use case 1: Opening and closing the activity

You start with the most basic use case, which is to start your app for the first time, then close the app down completely.

1. Compile and run the DessertClicker app, if it is not already running. As you've seen, the `onCreate()`, `onStart()`, and `onResume()` callbacks are called when the activity starts for the first time.

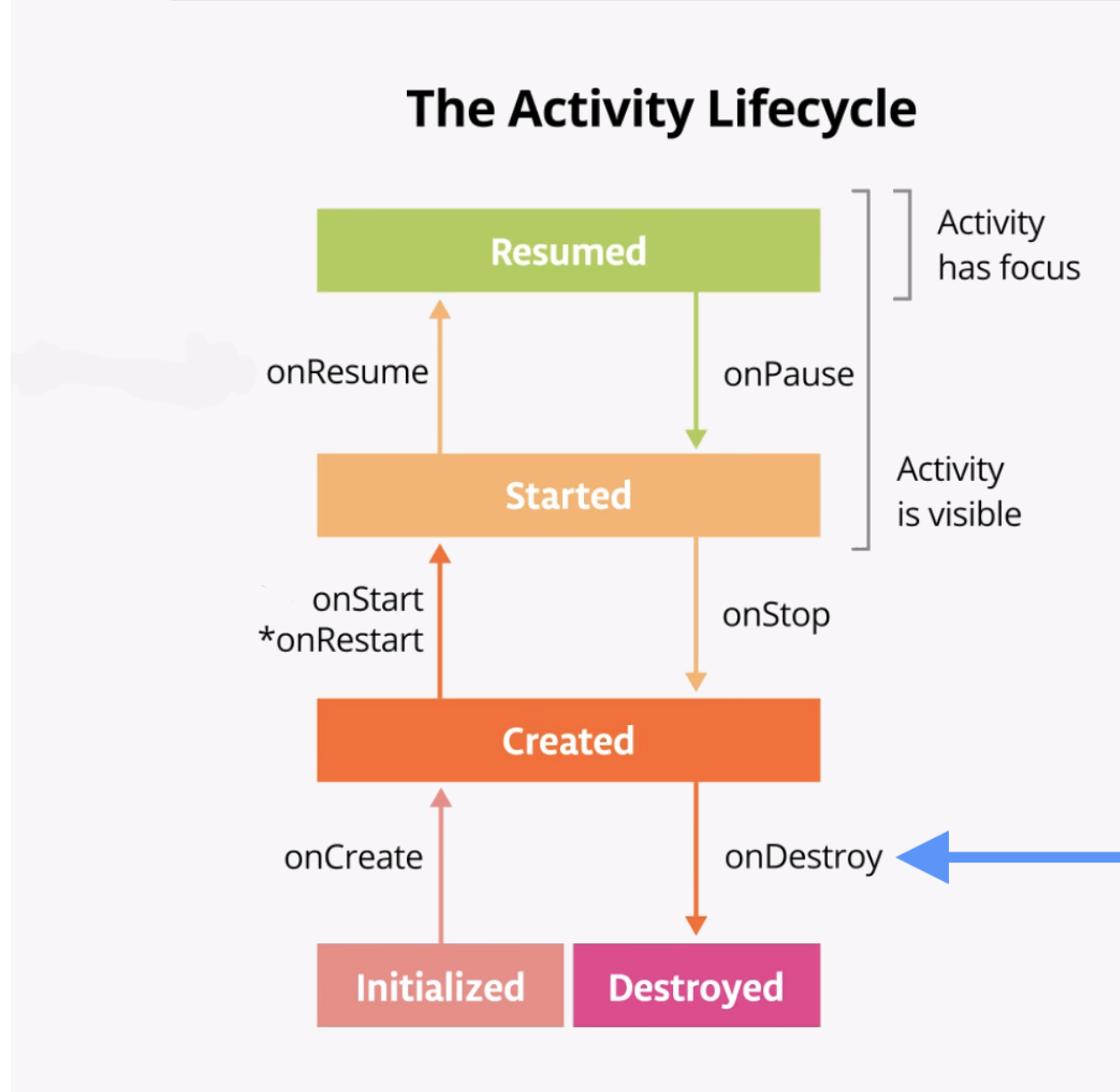
```
2020-10-16 10:27:33.244 22064-22064/com.example.android.dessertclicker
D/MainActivity: onCreate Called
2020-10-16 10:27:33.453 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStart Called
2020-10-16 10:27:33.454 22064-22064/com.example.android.dessertclicker
D/MainActivity: onResume Called
```

2. Tap the cupcake a few times.
3. Tap the **Back** button on the device. Notice in Logcat that `onPause()`, `onStop()`, and `onDestroy()` are called, in that order.

```
2020-10-16 10:31:53.850 22064-22064/com.example.android.dessertclicker
D/MainActivity: onPause Called
2020-10-16 10:31:54.620 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStop Called
2020-10-16 10:31:54.622 22064-22064/com.example.android.dessertclicker
D/MainActivity: onDestroy Called
```

In this case, using the **Back** button causes the activity (and the app) to be entirely closed. The execution of the `onDestroy()` method means that the activity was fully shut down and can be garbage-collected. [*Garbage collection*](#) refers to the automatic cleanup of objects that you'll no longer use. After `onDestroy()` is called, the system knows that those resources are discardable,

and it starts cleaning up that memory.



Your activity may also be completely shut down if your code manually calls the activity's `finish()` method, or if the user force-quits the app. (For example, the user can force-quit or close the app in the recents screen.) The Android system may also shut down your activity on its own if your app has not been on-screen for a long time. Android does this to preserve battery, and to allow your app's resources to be used by other apps.

4. Return to the DessertClicker app by finding all open apps on the [Overview screen](#). (Note this is also known as the Recents screen or recent apps.) Here's the Logcat:

```
2020-10-16 10:31:54.622 22064-22064/com.example.android.dessertclicker
D/MainActivity: onDestroy Called
```

```
2020-10-16 10:38:00.733 22064-22064/com.example.android.dessertclicker
D/MainActivity: onCreate Called
2020-10-16 10:38:00.787 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStart Called
2020-10-16 10:38:00.788 22064-22064/com.example.android.dessertclicker
D/MainActivity: onResume Called
```

The activity was destroyed in the previous step, so when you return to the app, Android starts up a new activity and calls the `onCreate()`, `onStart()`, and `onResume()` methods. Notice that none of the `DessertClicker` logs from the previous activity has been retained.

Note: The key point here is that `onCreate()` and `onDestroy()` are only called once during the lifetime of a single activity instance: `onCreate()` to initialize the app for the very first time, and `onDestroy()` to clean up the resources used by your app.

The `onCreate()` method is an important step; this is where all your first-time initialization goes, where you set up the layout for the first time by inflating it, and where you initialize your variables.

Use case 2: Navigating away from and back to the activity

Now that you've started the app and completely closed it, you've seen most of the lifecycle states for when the activity gets created for the first time. You've also seen all the lifecycle states that the activity goes through when it gets completely shut down and destroyed. But as users interact with their Android devices, they switch between apps, return home, start new apps, and handle interruptions by other activities such as phone calls.

Your activity does not close down entirely every time the user navigates away from that activity:

- When your activity is no longer visible on screen, this is known as putting the activity into the *background*. (The opposite of this is when the activity is in the *foreground*, or on screen.)
- When the user returns to your app, that same activity is restarted and becomes visible again. This part of the lifecycle is called the app's *visible* lifecycle.

When your app is in the background, it generally should not be actively running, to preserve system resources and battery life. You use the `Activity` lifecycle and its callbacks to know when your app is moving to the background so that you can pause any ongoing operations. Then you restart the operations when your app comes into the foreground.

In this step, you look at the activity lifecycle when the app goes into the background and returns again to the foreground.

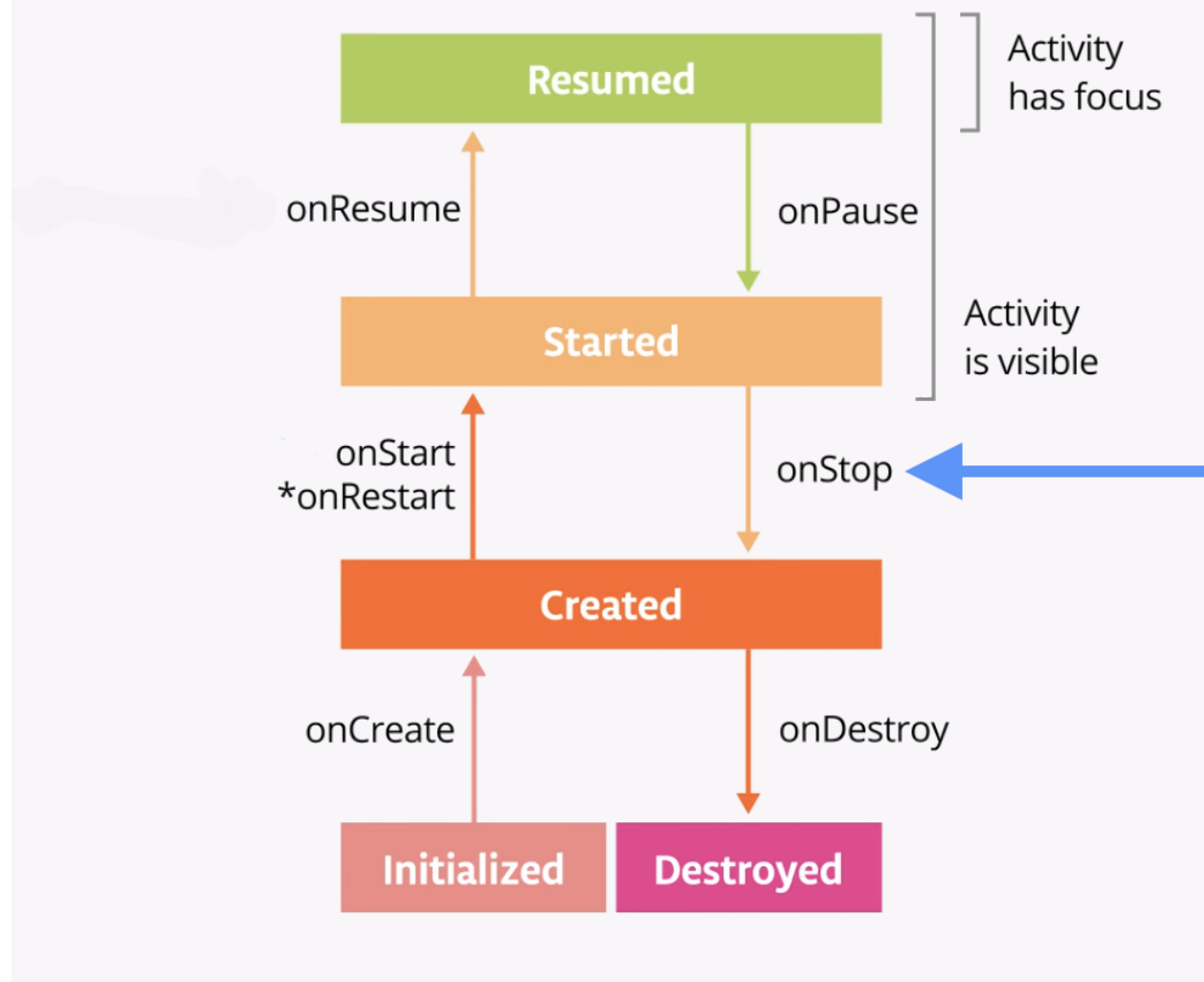
1. With the `DessertClicker` app running, click the cupcake a few times.
2. Press the **Home** button on your device and observe the Logcat in Android Studio. Returning to the home screen puts your app into the background rather than shutting

down the app altogether. Notice that the `onPause()` method and `onStop()` methods are called, but `onDestroy()` is not.

```
2020-10-16 10:41:05.383 22064-22064/com.example.android.dessertclicker
D/MainActivity: onPause Called
2020-10-16 10:41:05.966 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStop Called
```

When `onPause()` is called, the app no longer has focus. After `onStop()`, the app is no longer visible on screen. Although the activity has been stopped, the `Activity` object is still in memory, in the background. The activity has not been destroyed. The user might return to the app, so Android keeps your activity resources around.

The Activity Lifecycle



3. Use the recents screen to return to the app. Notice in Logcat that the activity is restarted with `onRestart()` and `onStart()`, then resumed with `onResume()`.

```
2020-10-16 10:42:18.144 22064-22064/com.example.android.dessertclicker
D/MainActivity: onRestart Called
2020-10-16 10:42:18.158 22064-22064/com.example.android.dessertclicker
D/MainActivity: onStart Called
2020-10-16 10:42:18.158 22064-22064/com.example.android.dessertclicker
D/MainActivity: onResume Called
```

When the activity returns to the foreground, the `onCreate()` method is not called again. The activity object was not destroyed, so it doesn't need to be created again. Instead of `onCreate()`, the `onRestart()` method is called. Notice that this time when the activity returns to the foreground, the **Desserts Sold** number is retained.

4. Start at least one app other than DessertClicker so that the device has a few apps in its recents screen.
5. Bring up the recents screen and open another recent activity. Then go back to recent apps and bring DessertClicker back to the foreground.

Notice that you see the same callbacks in Logcat here as when you pressed the Home button. `onPause()` and `onStop()` are called when the app goes into the background, and then `onRestart()`, `onStart()`, and `onResume()` when it comes back.

Note: The important point here is that `onStart()` and `onStop()` are called multiple times as the user navigates to and from the activity.

These methods are called when the app is stopped and moved into the background, or when the app is started again when it returns to the foreground. If you need to do some work in your app during these cases, then override the relevant lifecycle callback method.

So what about `onRestart()`? The `onRestart()` method is much like `onCreate()`. Either `onCreate()` or `onRestart()` is called before the activity becomes visible. The `onCreate()` method is called only the first time, and `onRestart()` is called after that. The `onRestart()` method is a place to put code that you only want to call if your activity is **not** being started for the first time.

Use case 3: Partially hide the activity

You've learned that when an app is started and `onStart()` is called, the app becomes visible on the screen. When the app is resumed and `onResume()` is called, the app gains the user focus, that is, the user can interact with the app. The part of the lifecycle in which the app is fully on-screen and has user focus is called the *interactive* lifecycle.

When the app goes into the background, the focus is lost after `onPause()`, and the app is no longer visible after `onStop()`.

The difference between focus and visibility is important because it is possible for an activity to be *partially* visible on the screen, but not have the user focus. In this step, you look at one case where an activity is partially visible, but doesn't have user focus.

1. With the DessertClicker app running, click the **Share** button in the top right of the screen.

The sharing activity appears in the lower half of the screen, but the activity is still visible in the top half.



11:18



Dessert Clicker



Today's Special



Share with



Messages



Bluetooth



Copy to clipboard

JUST ONCE

ALWAYS

2. Examine Logcat and note that only `onPause()` was called.

```
2020-10-16 11:00:53.857 22064-22064/com.example.android.dessertclicker
D/MainActivity: onPause Called
```

In this use case, `onStop()` is not called, because the activity is still partially visible. But the activity does not have user focus, and the user can't interact with it—the "share" activity that's in the foreground has the user focus.

Why is this difference important? The interruption with only `onPause()` usually lasts a short time before returning to your activity or navigating to another activity or app. You generally want to keep updating the UI so the rest of your app doesn't appear to freeze.

Whatever code runs in `onPause()` blocks other things from displaying, so keep the code in `onPause()` lightweight. For example, if a phone call comes in, the code in `onPause()` may delay the incoming-call notification.

3. Click outside the share dialog to return to the app, and notice that `onResume()` is called.

Both `onResume()` and `onPause()` have to do with focus. The `onResume()` method is called when the activity has focus, and `onPause()` is called when the activity loses focus.

5. Explore configuration changes

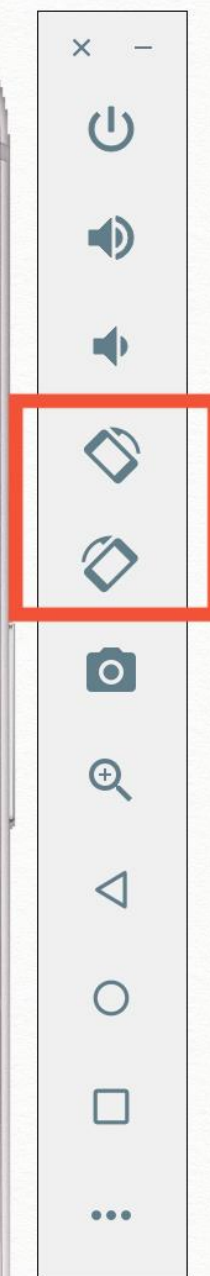
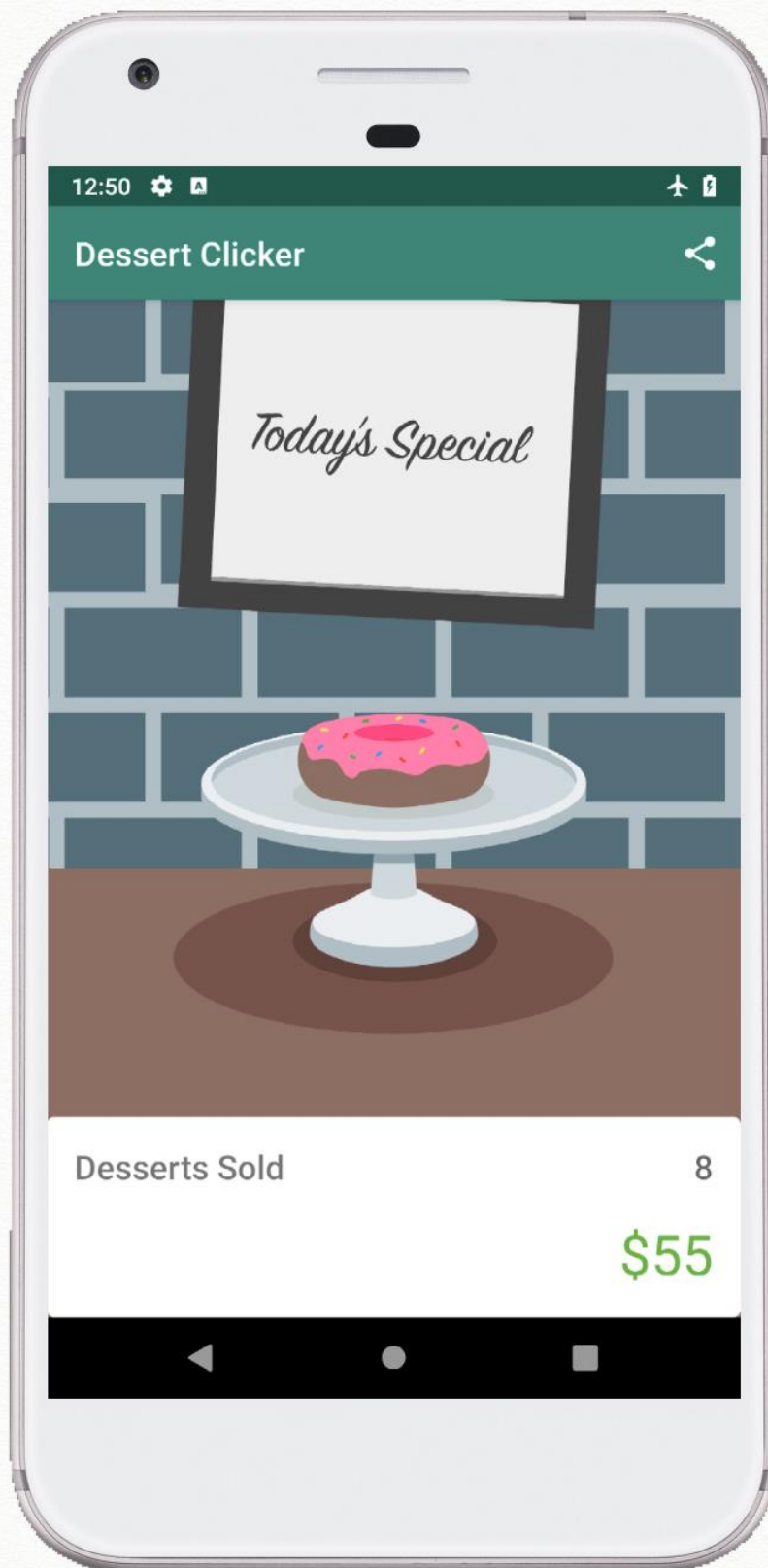
There's another case in managing the activity lifecycle that is important to understand: how configuration changes affect the lifecycle of your activities.

A *configuration change* happens when the state of the device changes so radically that the easiest way for the system to resolve the change is to completely shut down and rebuild the activity. For example, if the user changes the device language, the whole layout might need to change to accommodate different text directions and string lengths. If the user plugs the device into a dock or adds a physical keyboard, the app layout may need to take advantage of a different display size or layout. And if the device orientation changes—if the device is rotated from portrait to landscape or back the other way—the layout may need to change to fit the new orientation. Let's look at how the app behaves in this scenario.

Data loss on device rotation

1. Compile and run your app, and open Logcat.
2. Rotate the device or emulator to landscape mode. You can rotate the emulator left or right with the rotation buttons, or with the `Control` and arrow keys (`Command` and arrow keys

on a Mac).



3. Examine the output in Logcat. Filter the output on MainActivity.

```
2020-10-16 11:03:09.618 23206-23206/com.example.android.dessertclicker
D/MainActivity: onCreate Called
2020-10-16 11:03:09.806 23206-23206/com.example.android.dessertclicker
D/MainActivity: onStart Called
2020-10-16 11:03:09.808 23206-23206/com.example.android.dessertclicker
D/MainActivity: onResume Called
2020-10-16 11:03:24.488 23206-23206/com.example.android.dessertclicker
D/MainActivity: onPause Called
2020-10-16 11:03:24.490 23206-23206/com.example.android.dessertclicker
D/MainActivity: onStop Called
2020-10-16 11:03:24.493 23206-23206/com.example.android.dessertclicker
D/MainActivity: onDestroy Called
2020-10-16 11:03:24.520 23206-23206/com.example.android.dessertclicker
D/MainActivity: onCreate Called
2020-10-16 11:03:24.569 23206-23206/com.example.android.dessertclicker
D/MainActivity: onStart Called
```

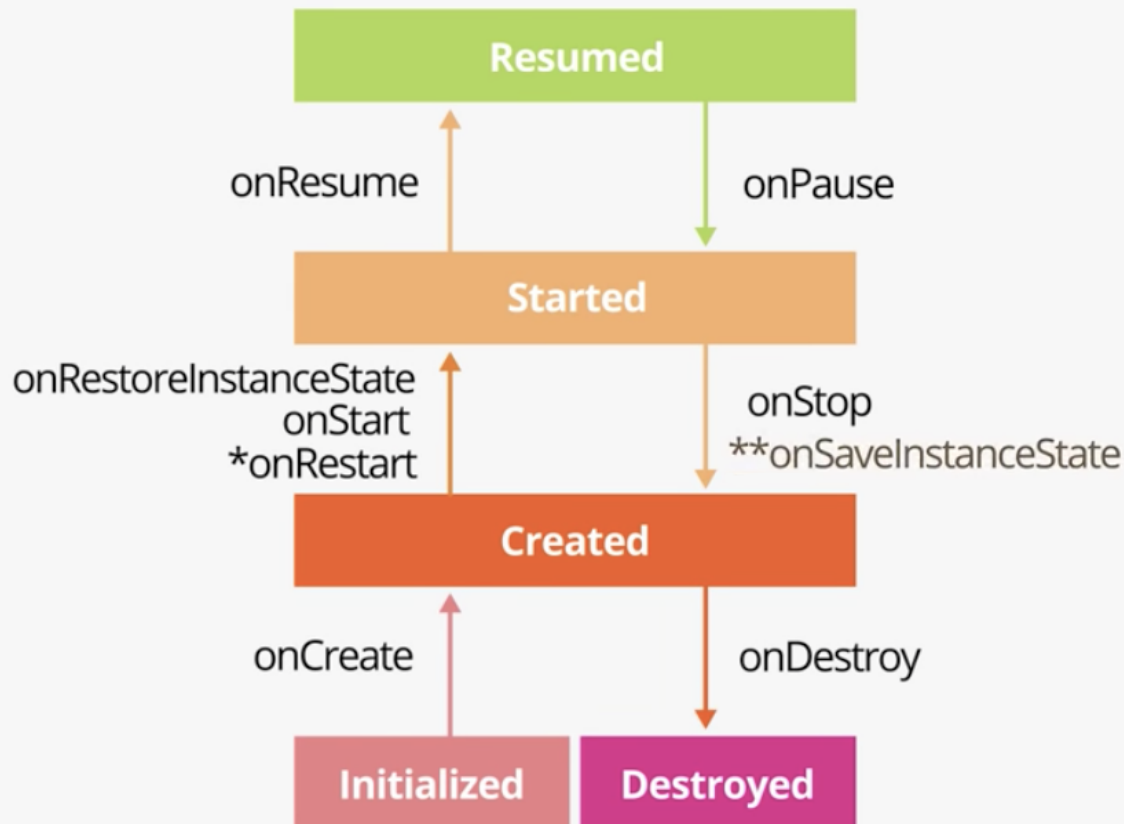
Notice that when the device or emulator rotates the screen, the system calls all the lifecycle callbacks to shut down the activity. Then, as the activity is re-created, the system calls all the lifecycle callbacks to start the activity.

4. When the device is rotated and the activity is shut down and re-created, the activity starts up with default values—the number of desserts sold and the revenue have reset to zeroes.

Use `onSaveInstanceState()` to save bundle data

The `onSaveInstanceState()` method is a callback you use to save any data that you might need if the `Activity` is destroyed. In the lifecycle callback diagram, `onSaveInstanceState()` is called after the activity has been stopped. It's called every time your app goes into the background.

The Activity Lifecycle



Think of the `onSaveInstanceState()` call as a safety measure; it gives you a chance to save a small amount of information to a bundle as your activity exits the foreground. The system saves this data now because if it waited until it was shutting down your app, the system might be under resource pressure.

Note: Sometimes Android shuts down an entire app process, which includes every activity associated with the app. Android does this kind of shutdown when the system is stressed and in danger of visually lagging, so no additional callbacks or code is run at this point. Your app's process is simply shut down, silently, in the background. But to the user, it doesn't look like the app has been closed. When the user navigates back to an app that the Android system has shut down, Android restarts that app. You'll want to ensure that the user doesn't experience any data loss when this happens.

Saving the data each time ensures that updated data in the bundle is available to restore, if it is needed.

1. In `MainActivity`, override the `onSaveInstanceState()` callback, and add a log statement.

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)

    Log.d(TAG, "onSaveInstanceState Called")
}
```

Note: There are two overrides for `onSaveInstanceState()`, one with just an `outState` parameter, and one that includes `outState` and `outPersistentState` parameters. Use the one shown in the code above, with the single `outState` parameter.

2. Compile and run the app, and click the **Home** button to put it into the background. Notice that the `onSaveInstanceState()` callback occurs just after `onPause()` and `onStop()`:

```
2020-10-16 11:05:21.726 23415-23415/com.example.android.dessertclicker
D/MainActivity: onPause Called
2020-10-16 11:05:22.382 23415-23415/com.example.android.dessertclicker
D/MainActivity: onStop Called
2020-10-16 11:05:22.393 23415-23415/com.example.android.dessertclicker
D/MainActivity: onSaveInstanceState Called
```

3. At the top of the file, just before the class definition, add these constants:

```
const val KEY_REVENUE = "revenue_key"
const val KEY_DESSERT_SOLD = "dessert_sold_key"
```

You will use these keys for both saving and retrieving data from the instance state bundle.

4. Scroll down to `onSaveInstanceState()`, and notice the `outState` parameter, which is of type `Bundle`. A [Bundle](#) is a collection of key-value pairs, where the keys are always strings. You can put simple data, such as `Int` and `Boolean` values, into the bundle. Because the system keeps this bundle in memory, it's a best practice to keep the data in the bundle small. The size of this bundle is also limited, though the size varies from device to device. If you store too much data, you risk crashing your app with the `TransactionTooLargeException` error.
5. In `onSaveInstanceState()`, put the revenue value (an integer) into the bundle with the `putInt()` method:

```
outState.putInt(KEY_REVENUE, revenue)
```

The `putInt()` method (and similar methods from the `Bundle` class like `putFloat()` and `putString()`) takes two arguments: a string for the key (the `KEY_REVENUE` constant), and the actual value to save.

6. Repeat the same process with the number of desserts sold:

```
outState.putInt(KEY_DESSERT_SOLD, dessertsSold)
```

Use onCreate() to restore bundle data

The Activity state can be restored in `onCreate(Bundle)` or [`onRestoreInstanceState\(Bundle\)`](#) (the `Bundle` populated by `onSaveInstanceState()` method will be passed to both lifecycle callback methods).

1. Scroll up to `onCreate()`, and examine the method signature:

```
override fun onCreate(savedInstanceState: Bundle?) {
```

Notice that `onCreate()` gets a `Bundle` each time it is called. When your activity is restarted due to a process shut down, the bundle that you saved is passed to `onCreate()`. If your activity was starting fresh, this `Bundle` in `onCreate()` is `null`. So if the bundle is not `null`, you know you're "re-creating" the activity from a previously known point.

Note: If the activity is being re-created, the `onRestoreInstanceState()` callback is called after `onStart()`, also with the bundle. Most of the time, you restore the activity state in `onCreate()`. But because `onRestoreInstanceState()` is called after `onStart()`, if you ever need to restore some state after `onCreate()` is called, you can use `onRestoreInstanceState()`.

2. Add this code to `onCreate()`, just after the binding variable is set:

```
if (savedInstanceState != null) {  
    revenue = savedInstanceState.getInt(KEY_REVENUE, 0)  
}
```

The test for `null` determines whether there is data in the bundle, or if the bundle is `null`, which in turn tells you if the app has been started fresh or has been re-created after a shutdown. This test is a common pattern for restoring data from the bundle.

Notice that the key you used here (`KEY_REVENUE`) is the same key you used for `putInt()`. To make sure you use the same key each time, it is a best practice to define those keys as constants. You use `getInt()` to get data out of the bundle, just as you used `putInt()` to put data into the bundle. The `getInt()` method takes two arguments:

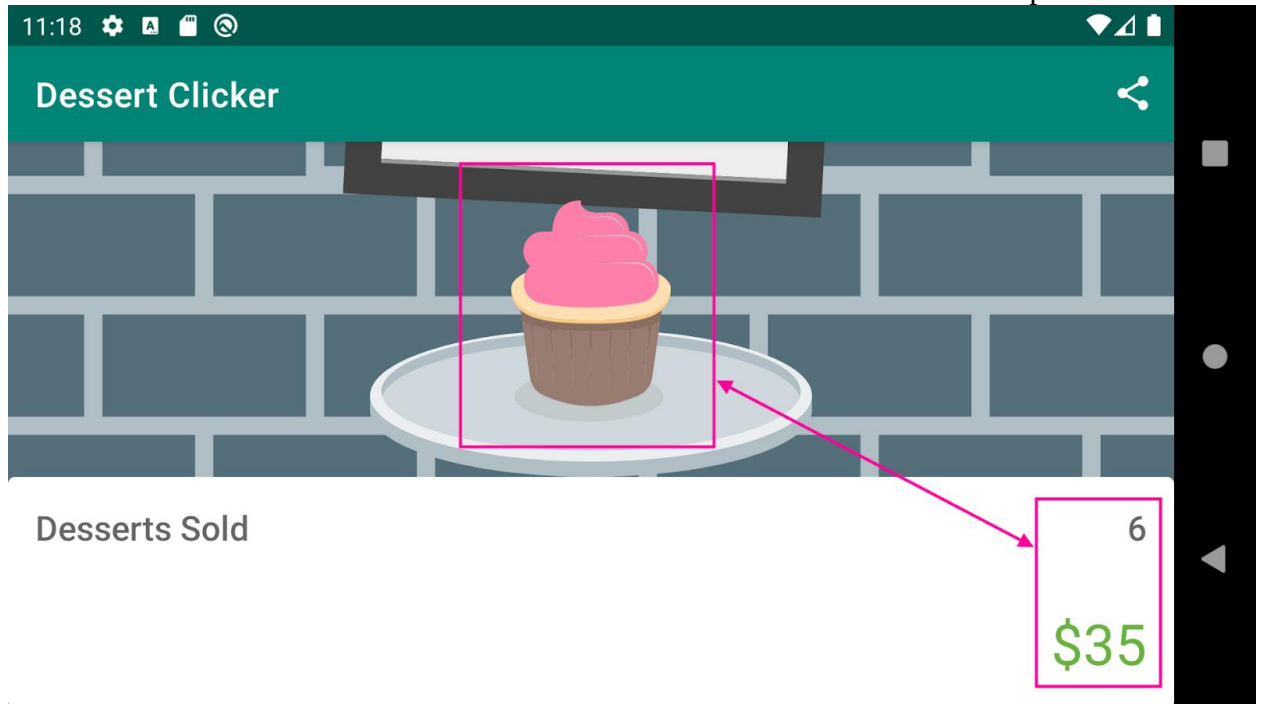
- A string that acts as the key, for example "key_revenue" for the revenue value.
- A default value in case no value exists for that key in the bundle.

The integer you get from the bundle is then assigned to the `revenue` variable, and the UI will use that value.

3. Add `getInt()` methods to restore the revenue and the number of desserts sold.

```
if (savedInstanceState != null) {  
    revenue = savedInstanceState.getInt(KEY_REVENUE, 0)  
    dessertsSold = savedInstanceState.getInt(KEY_DESSERT_SOLD, 0)  
}
```

4. Compile and run the app. Press the cupcake at least five times until it switches to a donut.
5. Rotate the device. Notice that this time the app displays the correct revenue and desserts sold values from the bundle. But also notice that the dessert has returned to a cupcake.



There's one more thing left to do to ensure that the app returns from a shutdown exactly the way it was left.

6. In `MainActivity`, examine the `showCurrentDessert()` method. Notice that this method determines which dessert image should be displayed in the activity based on the current number of desserts sold and the list of desserts in the `allDesserts` variable.

```
for (dessert in allDesserts) {  
    if (dessertsSold >= dessert.startProductionAmount) {  
        newDessert = dessert  
    }  
    else break  
}
```

This method relies on the number of desserts sold to choose the right image. Therefore, you don't need to do anything to store a reference to the image in the bundle in `onSaveInstanceState()`. In that bundle, you're already storing the number of desserts sold.

7. In `onCreate()`, in the block that restores the state from the bundle, call `showCurrentDessert()`:

```
if (savedInstanceState != null) {  
    revenue = savedInstanceState.getInt(KEY_REVENUE, 0)  
    dessertsSold = savedInstanceState.getInt(KEY_DESSERT_SOLD, 0)  
}
```

```
showCurrentDessert()  
}
```

8. Compile and run the app, and rotate the screen. Note now that both the values for desserts sold, total revenue, and the dessert image are correctly restored.

6. Summary

Activity lifecycle

- The *activity lifecycle* is a set of states through which an activity migrates. The activity lifecycle begins when the activity is first created and ends when the activity is destroyed.
- As the user navigates between activities and inside and outside of your app, each activity moves between states in the activity lifecycle.
- Each state in the activity lifecycle has a corresponding callback method you can override in your `Activity` class. The core set of lifecycle methods are:
[`onCreate\(\)`](#) [`onStart\(\)`](#) [`onPause\(\)`](#) [`onRestart\(\)`](#) [`onResume\(\)`](#) [`onStop\(\)`](#) [`onDestroy\(\)`](#)
- To add behavior that occurs when your activity transitions into a lifecycle state, override the state's callback method.
- To add skeleton override methods to your classes in Android Studio, select **Code** > **Override Methods** or press `Control+o`.

Logging with Log

- The Android logging API, and specifically the [`Log`](#) class, enables you to write short messages that are displayed in the Logcat within Android Studio.
- Use `Log.d()` to write a debug message. This method takes two arguments: the log *tag*, typically the name of the class, and the log *message*, a short string.
- Use the **Logcat** window in Android Studio to view the system logs, including the messages you write.

Preserving activity state

- When your app goes into the background, just after `onStop()` is called, app data can be saved to a bundle. Some app data, such as the contents of an `EditText`, is automatically saved for you.
- The bundle is an instance of [`Bundle`](#), which is a collection of keys and values. The keys are always strings.
- Use the `onSaveInstanceState()` callback to save other data to the bundle that you want to retain, even if the app was automatically shut down. To put data into the bundle, use the bundle methods that start with `put`, such as `putInt()`.
- You can get data back out of the bundle in the `onRestoreInstanceState()` method, or more commonly in `onCreate()`. The `onCreate()` method has a `savedInstanceState` parameter that holds the bundle.

- If the `savedInstanceState` variable is `null`, the activity was started without a state bundle and there is no state data to retrieve.
- To retrieve data from the bundle with a key, use the `Bundle` methods that start with `get`, such as `getInt()`.

Configuration changes

- A *configuration change* happens when the state of the device changes so radically that the easiest way for the system to resolve the change is to destroy and rebuild the activity.
- The most common example of a configuration change is when the user rotates the device from portrait to landscape mode, or from landscape to portrait mode. A configuration change can also occur when the device language changes or a hardware keyboard is plugged in.
- When a configuration change occurs, Android invokes all the activity lifecycle's shutdown callbacks. Then Android restarts the activity from scratch, running all the lifecycle startup callbacks.
- When Android shuts down an app because of a configuration change, it restarts the activity with the state bundle that is available to `onCreate()`.
- As with process shutdown, save your app's state to the bundle in `onSaveInstanceState()`.

7. Learn more

- [Log](#) class
- [Write and View Logs with Logcat](#)
- [Compile-time constants](#)
- [Activity](#) class
- [AppCompatActivity](#) class
- [Activity Developer Guide](#)